PRD Genie: architecture writeup

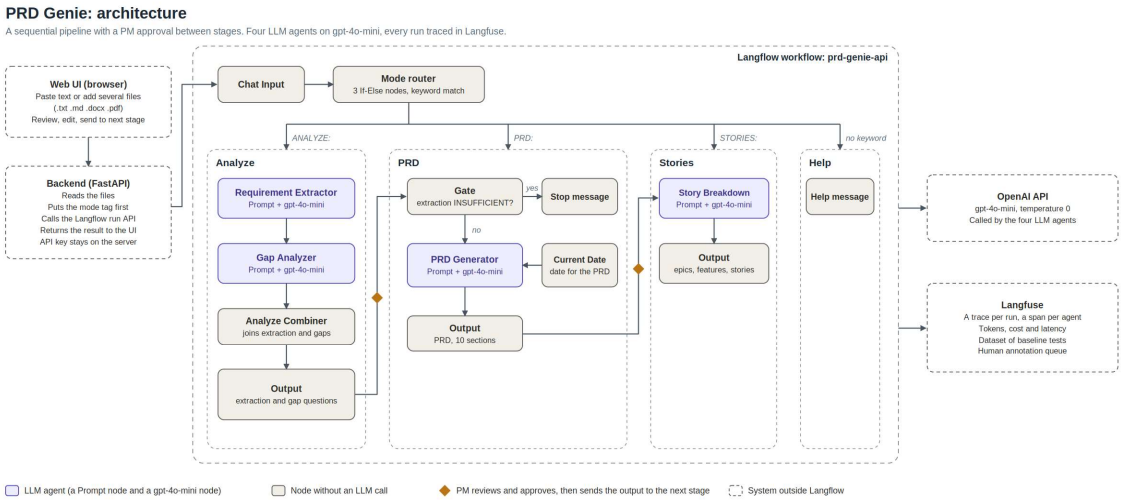
Prepared by: Naveen M R

Date: 2 November 2026

1. Problem summary

PMs and TPMs at NeuronForge spend much of their time turning meeting discussions into PRDs, epics and user stories by hand. The biggest wastes are rewriting transcripts into structured documents, inconsistent PRD formats between PMs, and requirements that get lost on the way. The biggest risk is an AI assistant that invents a requirement nobody stated, so every design choice below aims to keep each output item traceable to its source.

2. Architecture



The PM adds files or pasted text in a web UI; a mode router sends the request to one of three stages; the PM reviews and approves each result before running the next.

- Analyze: Requirement Extractor, then Gap Analyzer. Output: a structured extraction and clarification questions.
- PRD: a sufficiency gate, then the PRD Generator. Output: a 10-section PRD.
- Stories: the Story Breakdown agent. Output: epics, features and stories.

3. Tool selection

Category	Choice and why
Workflow platform	LangFlow. It builds multi-agent pipelines with custom prompts, has a deterministic router component, and exposes a run API that the UI calls. Used in class.

Category	Choice and why
LLM	gpt-4o-mini at temperature 0 for all four agents. It is cheap and fast enough for extraction and writing, and temperature 0 keeps before and after comparisons fair.
Inputs	Pasted text, or uploaded .txt, .md, .docx and .pdf files (several at a time), read by a small web UI. No OAuth is needed.
Output	Markdown, which matches the PRD template.
Observability	Langfuse, free tier. A trace per run with a span per agent, tokens, cost and latency. It also holds the dataset of the 12 tests and a human annotation queue.

4. Orchestration pattern

The pattern is a sequential pipeline (Analyze, PRD, Stories) with human approval between stages, entered through a keyword router.

- Every input follows the same order, so a router that picks between specialist agents, or a hierarchy of sub-agents, would add complexity without a benefit.
- The stages run separately because the PM approves and may edit each result first. Stories should come from the approved PRD, not from an AI draft, and a long transcript is processed only once, which keeps cost down.
- The router matches a mode keyword (ANALYZE:, PRD:, STORIES:) with a plain text rule, not an LLM. It costs nothing and cannot misroute.
- A deterministic gate stops the PRD stage when the extraction is marked INSUFFICIENT, so a PRD is never built from nothing.
- Langflow's Human Input pause was tested. It offers buttons only, with no typed answers, so a questions-first step stays on the roadmap.

5. Agents

Agent	Input and output	Key prompt rules
Requirement Extractor	Transcript, notes or brief to a JSON extraction: requirements with verbatim quotes, criteria, stakeholders, deadlines, dependencies, ambiguous items, contradictions, risks and a sufficiency flag	Every requirement needs an exact quote. Never invent requirements, numbers, dates or risks. Copy numbers exactly. Priority words and deadlines only if they are in the input. Vague input is INSUFFICIENT, and the missing items are named.
Gap Analyzer (extended	Extraction to clarification questions with severity and a	One question per gap, with evidence. No gap for information already stated. Contradictions and

Agent	Input and output	Key prompt rules
capability)	suggested owner	unknown ETAs are High. Owner only if named.
PRD Generator	Approved extraction to a 10-section PRD	Use only the extraction. Unknown fields read “Not specified in source”. Priority only from stated signals. Date and version supplied by the system. Personas are user types only.
Story Breakdown	Approved PRD to epics, features and stories	One story per capability and per persona. Criteria copied verbatim in quotes. Benefit is a placeholder unless stated. No drafted business rules. Every story cites an FR or NFR id. INVEST is applied silently.

6. Cost analysis

Measured in Langfuse on one T1 chain. Every step’s cost matches the gpt-4o-mini price of $0.15 per million input tokens and $0.60 per million output tokens.

Stage	Time	Tokens	Cost
Analyze (Extractor and Gap)	9.07 s	2,539	$0.00053
PRD	7.90 s	1,817	$0.00045
Stories	19.39 s	3,816	$0.00118
Full chain	36.4 s	8,172	$0.0022

A document lifecycle (one Analyze, two PRD revisions, one Stories) costs about $0.0026. At three lifecycles per PM per day (an assumption), that is about $0.008 per day and $0.17 per month. Stories is 55% of the cost. Output tokens are 25% of all tokens but 58% of the cost. A 10,000-token transcript would add about $0.0015 to the Analyze stage, which makes long inputs the main cost risk.

7. Evaluation strategy

Three layers are used. First, a baseline set of 12 tests from the problem statement, run as 17 stage runs and checked against each test’s expected items in a results tracker. Second, Langfuse traces for per-agent output, cost and latency. Third, human annotation in a Langfuse queue, where nine traces (the T1 chain and T2 to T7) were scored for faithfulness, completeness, hallucination, usefulness and flag handling. Faithfulness averaged 4.9 out of 5, completeness 4.7 and usefulness 3.3, and one trace (T6) contained an unsupported detail.

Production metrics: extraction completeness (target 90% or more), hallucination rate (items not traceable to the source and not tagged as drafts; target 2% or less), format compliance (target 100%), and PM time saved per PRD (target 50% less, to be measured in a pilot).

8. Testing observations

Result: 11 of 12 baseline tests meet their criteria. T3 fails. One of 17 runs contained a small unsupported detail (a role expanded from PM to Product Manager in T6). The human scores agree: the agents are grounded but not yet fully useful, because they sometimes fail to flag or capture something (T3, T5, T7).

What worked:

- Vague input was flagged INSUFFICIENT and not turned into requirements (T2, T5, T9), and the gate stopped T9 before any PRD was written.
- Exact figures survived: 10,000 concurrent users, 200 ms at p95, Salesforce REST API v52 (T7).
- An explicit disagreement was found and raised as a High gap (T6).
- Acceptance criteria were copied verbatim and attached to separate stories (T4), and personas stayed separate (T8).

What failed, and what was fixed:

Agent	Failure	Fix	Result
Extractor	Invented a risk and a priority copied from schema examples; invented a deadline (Q3) in T8	Removed the examples; rules to copy only words that appear in the input and to list deadlines only if present	Gone in the final runs
Extractor	Insufficient input gave no useful question (T9)	Rule that INSUFFICIENT must name what is missing	The Gap agent now asks what is needed
PRD Generator	Persona was the PM; placeholder date; a requirement id in the target column	Rules on personas and targets; a Current Date node	PRD matches the template
Story Breakdown	A long prompt invented defaults, message texts and a CSV upload (T4, T8, T12); criteria were paraphrased; benefits and priorities were invented	A strict prompt: criteria in quotes, a benefit placeholder, priority only from the PRD column	T4, T8 and T12 pass with no invented specifics

Known limitations:

- An implicit trade-off inside one statement (T3: frequent refresh versus fewer API calls) is not detected. Explicit disagreements are.
- Output varies between runs even at temperature 0. In one T6 run the extraction flipped to INSUFFICIENT, and a T4 PRD lost its persona.
- The PRD sometimes repeats acceptance criteria as functional requirements.
- Stories are thin: business rules and validations need PM answers, so a questions-first step is the planned fix.

- The Gap agent does not ask which thing is meant when the input names something without identifying it (which competitor, which API, which dashboard).